```
transfer.cgi">
<input type="hidden" name="from" value="35367021">
<input type="hidden" name="to" value="48412334">
<input type="hidden" name="amount" value="5000">
<input type="hidden" name="date" value="05072010">
</form>

<script>document.badform.submit()</script>
```

The CSRF attack using the HTTP POST method is summarised in Figure 2. Please note that this scenario was used just to explain CSRF clearly, and that no bank is careless enough to let such an attack happen. I reiterate that you should not try any of these steps on any public server.

Scenario 2

Another CSRF attack scenario exploits the firewall Web management system. This is again based on session cookies. Let's suppose the firewall Web management application has a function that allows an authenticated user to delete a rule, specified by its positional number, or all the rules of the configuration, if the user enters '*'. For example, a valid URL to delete a single rule, number 5, would be:

```
http://www.example.com/firemange/delete?rule=5
```

Malicious HTML for the CSRF, using HTTP GET is as follows:

```
<HTML>
<IMG SRC="http://www.example.com/firemange/delete?rule=5"
width="0" height="0">
</HTML>
```

Scenario 3

Routers for private use, like popular DSL routers, come with a preconfigured IP address for the LAN interface (e.g., 192.168.0.1), which most users do not change. Thus, the host-part of the URL is known in most cases. Attackers are familiar with the workings and URLs of most popular routers, and if they can social-engineer the victim into clicking a malicious link, this will (for example) enable the remote Web management on port 8080, allowing it to be administered from any computer on the Internet. The malicious Web page might contain the following HTML:

```
<IMG SRC=http://192.168.0.1/enable_remote_management=true&ips_
allowed="&mgmtport=8080>
```

The attacker can also change the router's default account name, and could also compromise network printers—so if you have a DSL/home router left at its default settings, change the preconfigured IP and default passwords right now!

Using well-known URLs for Web applications, an attacker can also reset your Web-based account passwords, tamper with your corporate Web-mail, and delete or modify accounts in Web applications. CSRF is a serious vulnerability, and cannot be taken lightly.

The differences between XSS and CSRF

Though CSRF seems similar to Cross-Site Scripting (XSS) at first, both are completely different attack vectors. Where XSS aims at inserting active code in an HTML document to either abuse client-side active scripting holes, or to send privileged information (e.g., authentication/session cookies) to an unknown evil site, CSRF aims to perform unwanted actions on a site where the victim has some prior relationship and authority. Moreover, where XSS sought to steal your online trading cookies so an attacker could manipulate a victim's account, CSRF seeks to use the victims' cookies to force them to execute a trade without their knowledge or consent. While XSS attacks exploits the trust that a user has on the website, CSRF attacks exploit the trust that the website has in its user.

Types of CSRF attacks

CSRF attacks can be divided into two major categories—reflected and stored/local.

Reflected CSRF attacks

In a reflected CSRF attack, the attacker uses a system outside the application to expose the victim to the exploit link or content. This can be done using a blog, an email message, an instant message, a message-board posting, or even a flyer posted in a public place with a URL that a victim types in.

Reflected CSRF attacks will frequently fail, as users may not be currently logged into the target system when the exploits are tried. The trail from a reflected CSRF attack, however, may be under the attacker's control, and could be deleted once the exploit is completed. The three attack scenarios we looked at earlier are examples of reflected CSRF attacks.

Local/stored CSRF attacks

A stored/local CSRF attack is one where the attacker can use the application itself to provide the victim the exploit link, or other content which directs the victim's browser to perform attacker-controlled actions in the application. Stored CSRF vulnerabilities are more likely to succeed, since the user who receives the exploit content is almost certainly currently authenticated to perform actions.

Stored CSRF attacks also have a more obvious trail, which may lead back to the attacker, since the origin of the malicious HTTP request is hosted in the attacked website. Examples include bulletin boards and social sites where users are allowed to post images with foreign URL sources. These are harder to find and destroy.

Why CSRF works

To explain the root causes of, and solutions to CSRF attacks,